Parallel Sparse Triangular Solve

15-418 Final Proposal

Kelly Zhang (kellyzha@andrew.cmu.edu)
https://github.com/kllyzhang/15418-project

1 Summary

We will implement parallel scheduling strategies for sparse triangular solves (SpTRSV) on multicore CPUs using C++ and OpenMP. It will compare sequential execution, level-synchronous scheduling, and dependency-aware scheduling using synchronization sparsification and barrier-list techniques from recent research. The evaluation will examine synchronization overhead, workload imbalance, preprocessing costs, cache behavior, and scalability across different sparse matrix structures.

2 Background

Sparse triangular solve is an important computational kernel used in sparse matrix factorization, preconditioning, and scientific computing. For example, it commonly appears in solving linear systems through LU and Cholesky factorizations.

Given a sparse lower-triangular matrix $\mathbf{L} \in \mathbb{R}^{n \times n}$ and a dense vector $\mathbf{b} \in \mathbb{R}^n$, we want to solve linear system $\mathbf{Lx} = \mathbf{b}$ for $\mathbf{x} \in \mathbb{R}^n$ using forward substitution. SpTRSV offers opportunities for parallelism as dependencies form a directed acyclic graph (DAG), allowing independent rows to be computed simultaneously.

The standard sequential compressed sparse row (CSR) forward-substitution algorithm is shown,

1: **for** $i = 0, \ldots, n-1$ **do**
2: $\quad t \leftarrow b_i$
3: $\quad$ **for** $j = 0, \ldots, i-1$ when $L_{i,j} \neq 0$ **do**
4: $\quad\quad t \leftarrow t - L_{i,j} \cdot x_j$
5: $\quad$ **end for**
6: $\quad x_i \leftarrow \frac{t}{L_{i,i}}$
7: **end for**

A common parallel approach is level-synchronous scheduling, which executes independent rows in parallel but requires synchronization between levels. More advanced approaches, such as barrier-list scheduling, aim to reduce synchronization overhead and improve workload balance. Our project will compare these scheduling strategies to investigate how dependency structures and scheduling decisions affect parallel performance.

3 Challenge

The main challenge in parallelizing SpTRSV is managing irregular dependencies while minimizing synchronization overhead and maintaining balanced workloads.

- **Irregular dependencies**: Each row depends on a different set of preceding rows, forming a DAG. This limits which rows can execute concurrently, and matrices with long dependency chains may offer little parallelism.
- **Synchronization overhead**: Level-synchronous scheduling requires all threads to finish a level before proceeding to the next. This can cause unnecessary waiting, while finer-grained dependency tracking introduces additional synchronization and atomic operation costs.
- **Workload imbalance**: Different rows contain different numbers of nonzero entries and therefore require different amounts of computation. Uneven work distribution can leave some threads idle while others are still processing.
- **Memory locality**: CSR storage provides efficient row-wise access, but accessing previously computed solution values involves irregular memory reads. Distributing rows across threads may reduce cache locality and increase memory-access overhead.
- **Scheduling and preprocessing costs**: More advanced scheduling algorithms require additional computation to analyze dependencies and assign work to threads. These preprocessing costs may outweigh execution-time improvements for a single solve, but can be amortized when solving multiple systems with the same matrix.

Our goal is to understand how these factors interact and determine when more sophisticated scheduling strategies outperform simpler level-synchronous execution.

4 Resources

We will implement our code base from scratch.

Computing Resources

- C++ with OpenMP
- GHC cluster and Pittsburgh Supercomputing Center (PSC) machines

Data

- SuiteSparse Matrix Collection
- Synthetic matrices

References

- Bohnlein et al. (2025): *Efficient Parallel Scheduling for Sparse Triangular Solvers.* arxiv.org/abs/2503.05408: Investigates barrier-list scheduling, workload balancing, and synchronization reduction.
- Park et al. (2014) *Sparsifying Synchronization for High-Performance Shared-Memory Sparse Triangular Solver.* doi.org/10.1007/978-3-319-07518-1_8: Introduces dependency sparsification and point-to-point synchronization to reduce overhead in parallel SpTRSV.

5 Goals & Deliverables

We PLAN TO ACHIEVE the following,

- **Sequential baseline**: Implement a correct sequential SpTRSV solver using CSR format to establish a performance baseline and verify numerical correctness.
- **Level-synchronous parallel solver**: Implement an OpenMP solver that groups independent rows into levels and executes each level in parallel, with global barriers between levels and test against the sequential solver using synthetic and real-world sparse matrices, verifying numerical accuracy and correct dependency ordering.
- **Synchronization-sparsified solver**: Implement the core approach from Park et al. (2014), reducing redundant synchronization dependencies and using fine-grained synchronization to enable rows to execute once their required predecessors have completed.
- **Barrier-list scheduling**: Implement a simplified barrier-list scheduling approach inspired by Bohnlein et al. (2025), exploring whether optimized barrier placement and workload-aware processor assignment improve performance over conventional level scheduling and synchronization sparsification.
- **Performance evaluation**: Measure execution time, speedup, parallel efficiency, cache behavior, and preprocessing overhead across different matrix sizes, dependency structures, and thread counts. Investigate how dependency depth, synchronization overhead, workload imbalance, and memory locality limit parallel performance. Compare preprocessing costs against execution-time savings, including repeated solves using the same matrix. Compare speedups for real-world and synthetic matrices across our different implementations. Verify numerical correctness against known solutions.

We HOPE TO ACHIEVE the following,

- **Further scheduling optimizations**: Experiment with workload-aware scheduling and thread assignment to improve load balance and reduce idle time.

If progress is slower than expected, we will prioritize correct sequential, level-synchronous, and synchronization-sparsified implementations, along with correctness testing and performance evaluation.

6 Platform Choice

We plan to implement our project in C++ using OpenMP on multicore CPUs, utilizing the GHC cluster and Pittsburgh Supercomputing Center (PSC) machines. These platforms provide access to parallel computing resources for evaluating SpTRSV algorithms across different thread counts and matrix sizes. OpenMP supports parallel task scheduling and synchronization, allowing us to investigate the effects of dependency constraints, barrier overhead, workload imbalance, and memory locality. We will use the available machines to measure scalability and compare the performance of different scheduling strategies under controlled experimental conditions.

7 Schedule

Time	Items
WEEK 1 (11/09 - 11/15)	• Proposal Report Submission • Study all cited papers and plan implementation • Prepare synthetic matrices and SuiteSparse datasets • Implement sequential CSR SpTRSV solver
WEEK 2 (11/16 - 11/22)	• Implement level-synchronous OpenMP solver • Validate correctness and dependency levels • Conduct preliminary scaling experiments • Begin Park et al. synchronization sparsification solver implementation
WEEK 3 (11/23 - 11/29)	• Milestone Report Submission • Complete synchronization sparsification implementation • Validate implementation correctness
WEEK 4 (11/30 - 12/06)	• Implement Bohnlein et al. barrier-list scheduling • Run matrix benchmarks, performance profiling and analysis • [stretch] Explore further scheduling optimizations
WEEK 5 (12/07 - 12/11)	• Complete experiments and performance comparisons • Final Report Submission • Poster Presentation